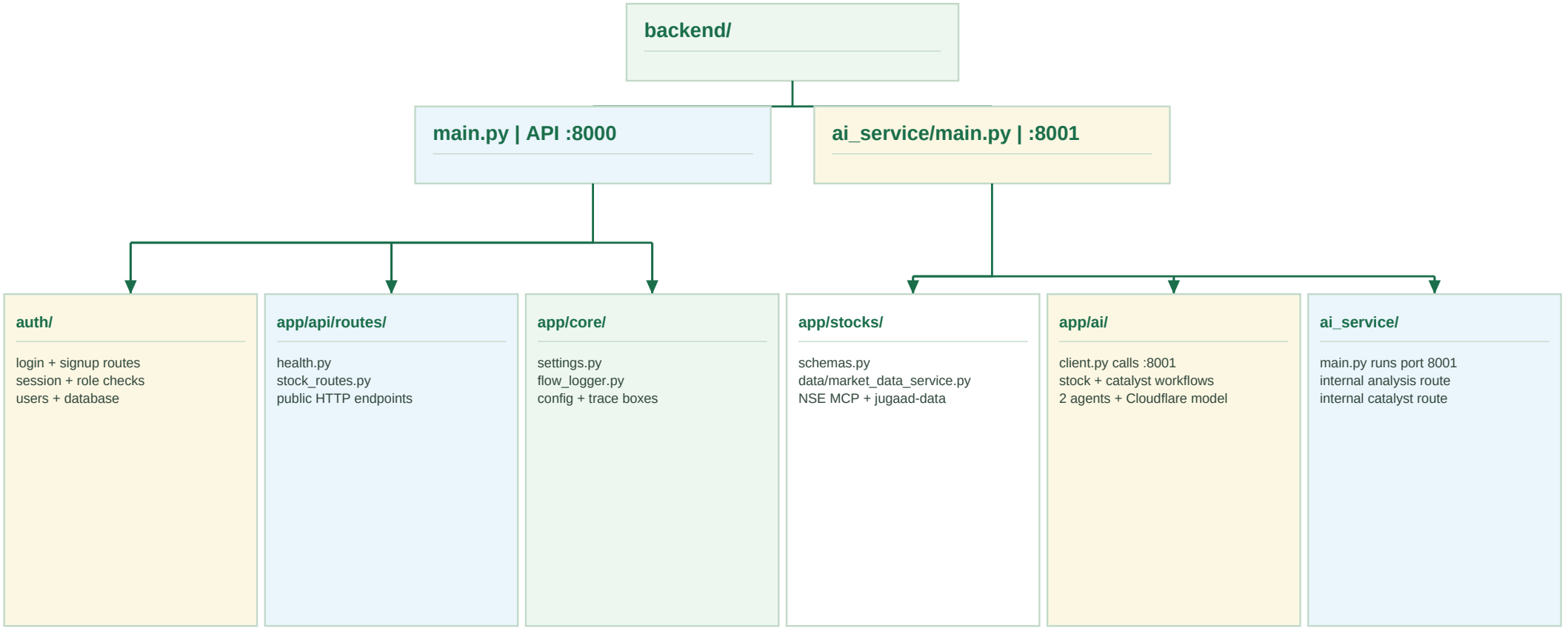
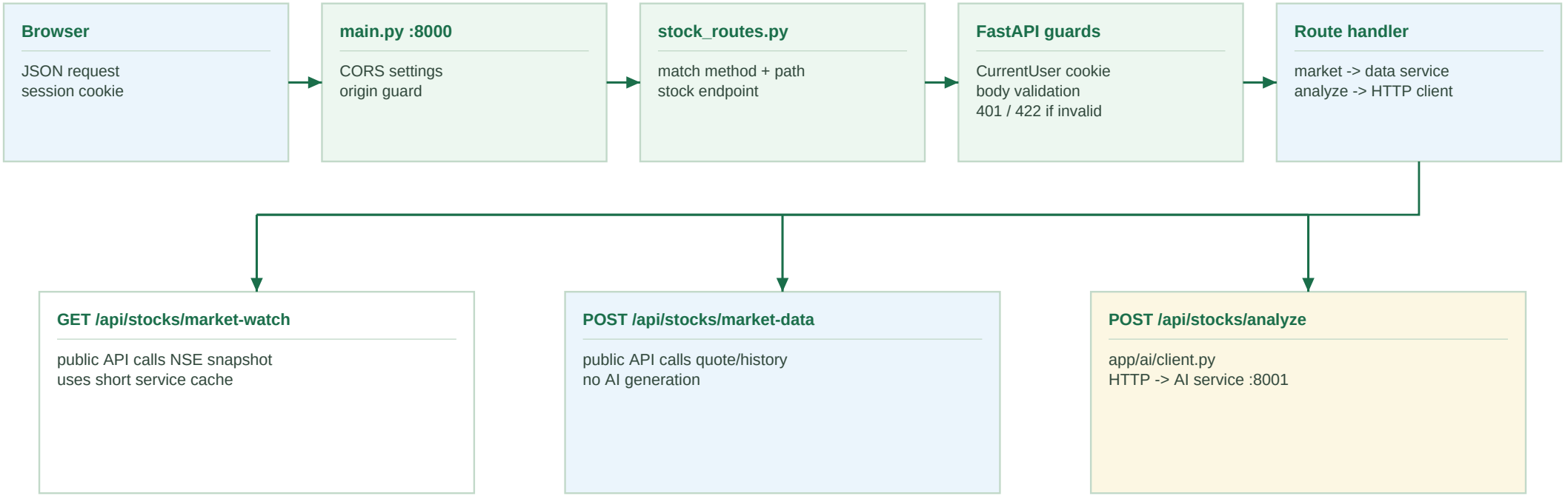
BACKEND | TWO SERVICE FOLDER MAP



Follow one analysis request

Browser -> public API :8000 -> app/ai/client.py -> AI service :8001 -> LangGraph -> NSE MCP/jugaad-data + Stock Agent/Cloudflare -> typed result back to API.
Market-watch and quote endpoints stay in the public API process. The analysis call waits for its worker response; moving the work to another process isolates the API from the model and NSE execution.

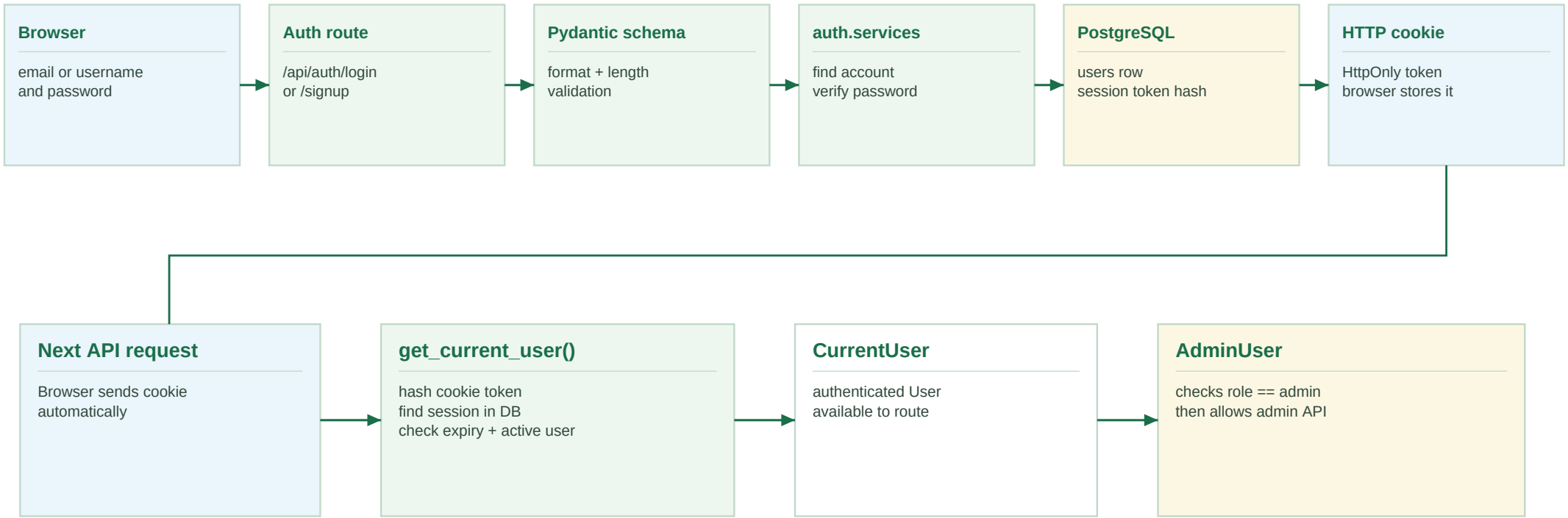
BACKEND | REQUEST ROUTING AND ACCESS



Why the API stays responsive

Every route first checks the login session and validates its request. The analysis route then awaits an async HTTP call to port 8001; its event loop can serve other requests while the worker is busy.
The browser still waits for analysis to finish. This separates process work and failures; it does not make a slow model response instant.

BACKEND | AUTH, SESSION, AND ROLES



Sign out

POST /api/auth/logout deletes the DB session
and clears the browser cookie

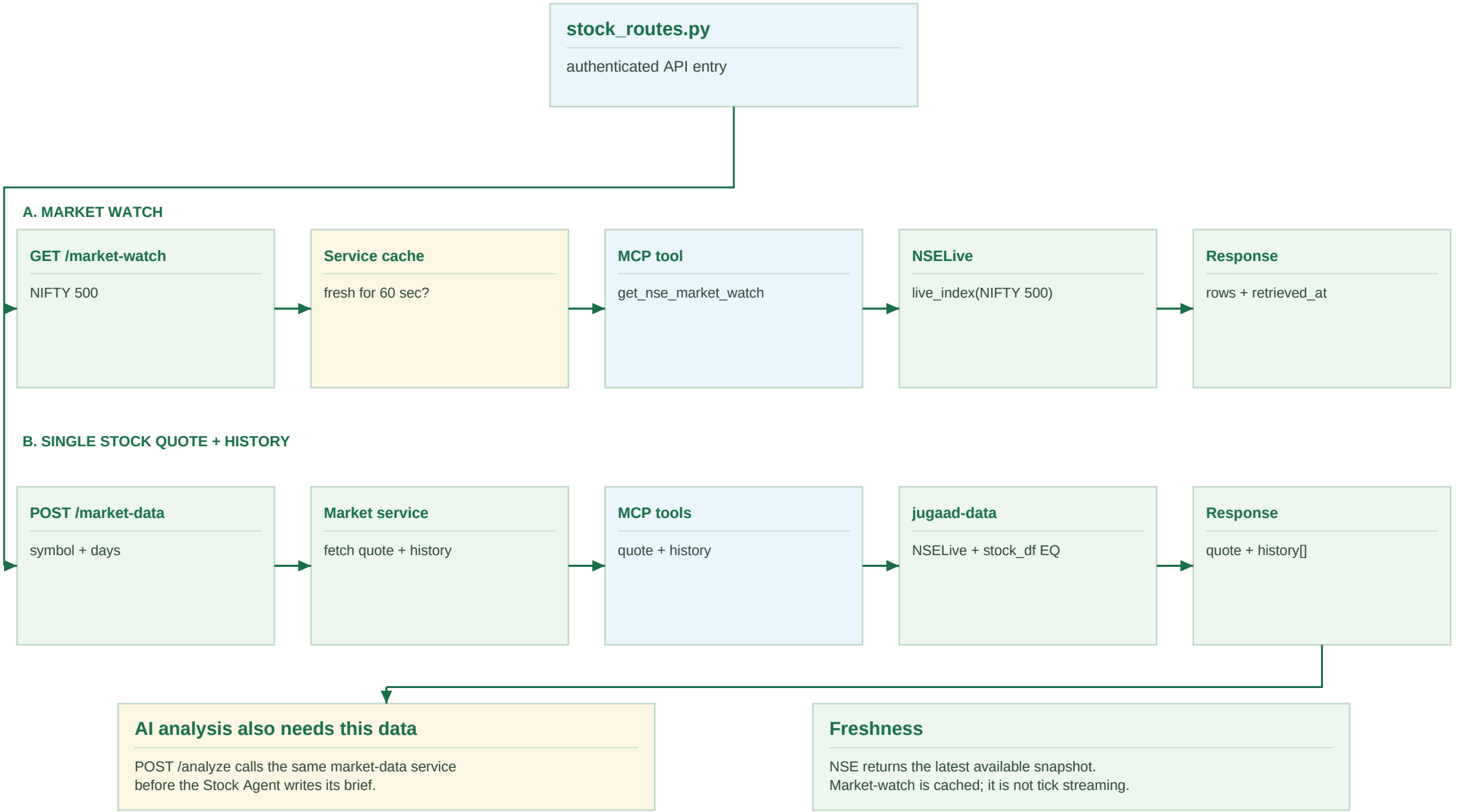
Role defaults

Public signup creates role=user.
The admin CLI creates the first admin account.

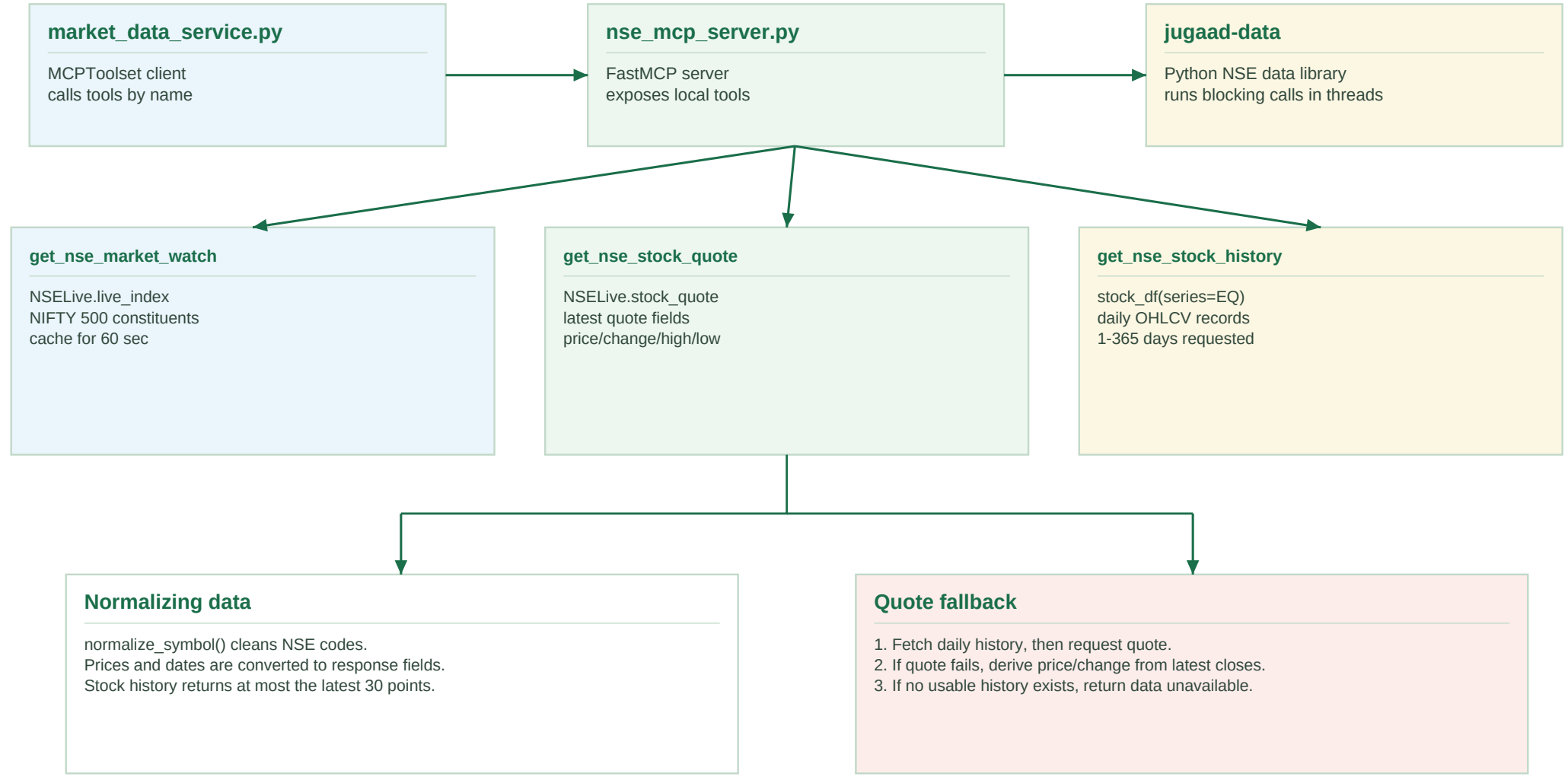
Why the database stores a hash

The browser receives a random session token, but PostgreSQL stores only its SHA-256 hash. On later requests, the backend hashes the cookie and looks up that hash.
The password itself is never stored; auth.services.py stores a password hash and verifies it at login.

BACKEND | MARKET DATA ENDPOINTS



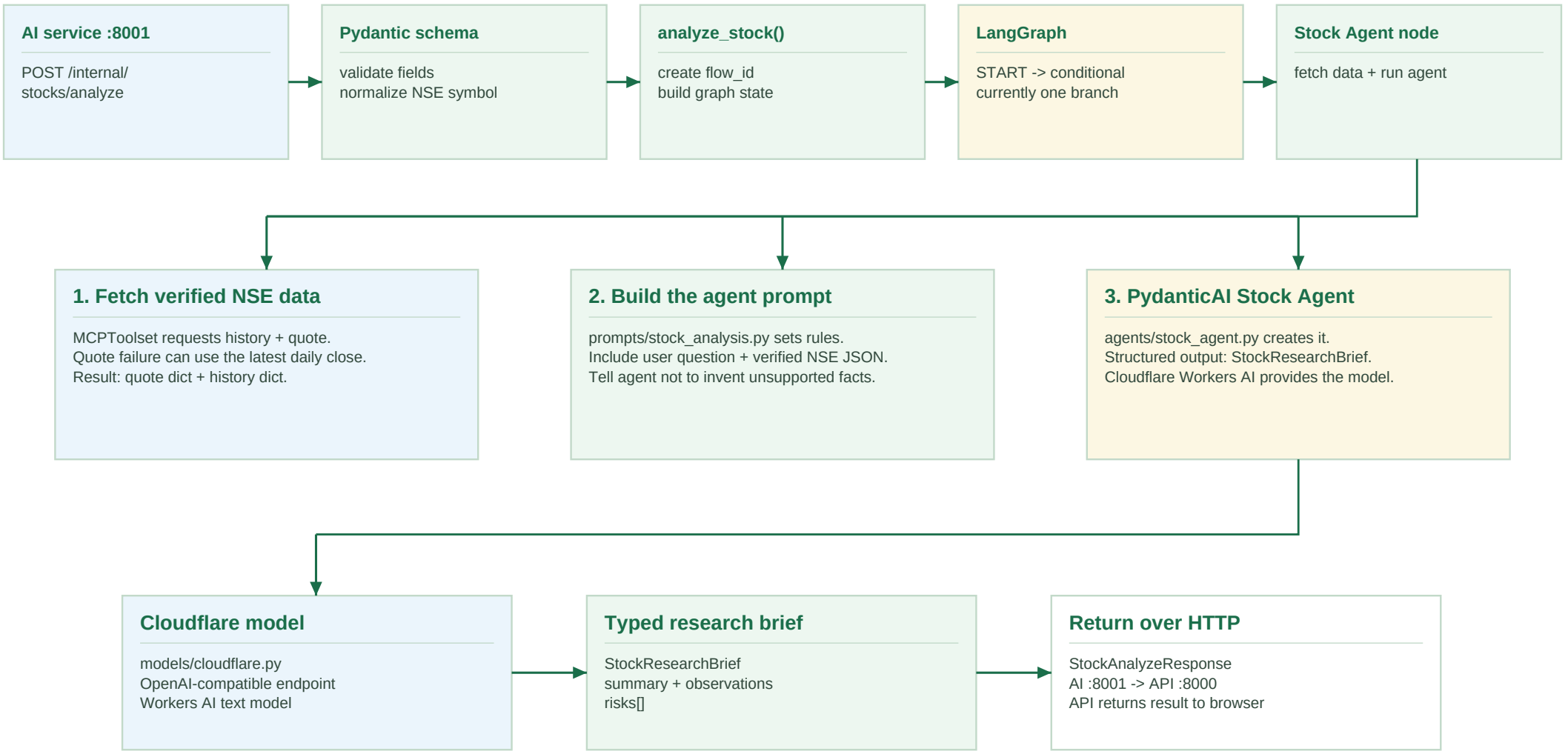
BACKEND | MCP TOOLS AND NSE DATA



MCP in this project

MCP is the tool boundary between the app and stock-data functions. FastMCP defines the tools; PydanticAI's MCPToolset calls them.
NSE daily history is not an intraday tick feed. The market-watch endpoint gives the latest available snapshot and caches it briefly.

BACKEND | AI SERVICE, LANGGRAPH, AND STOCK AGENT



This diagram runs inside ai_service/main.py on port 8001

The public API waits for the typed result. Graph steps, NSE/MCP calls, and the Cloudflare request run in this worker process. Other API routes run in their own process, subject to normal machine resource limits.

BACKEND | STATE, RESPONSE, AND ERRORS

LangGraph state

Input: symbol
Input: question + history_days
Input: flow_id
Node adds: quote + history + brief

StockResearchBrief

summary: plain-language overview
key_observations: string list
risks: string list
PydanticAI enforces this shape

StockAnalyzeResponse

status + symbol + question
quote + daily history[]
brief + informational disclaimer
FastAPI response model

Common API failures

401 | no valid login session
403 | blocked origin or non-admin access
409 | duplicate email/username on signup
422 | request fields failed validation
503 | AI service unavailable/timed out, or NSE/Cloudflare failed

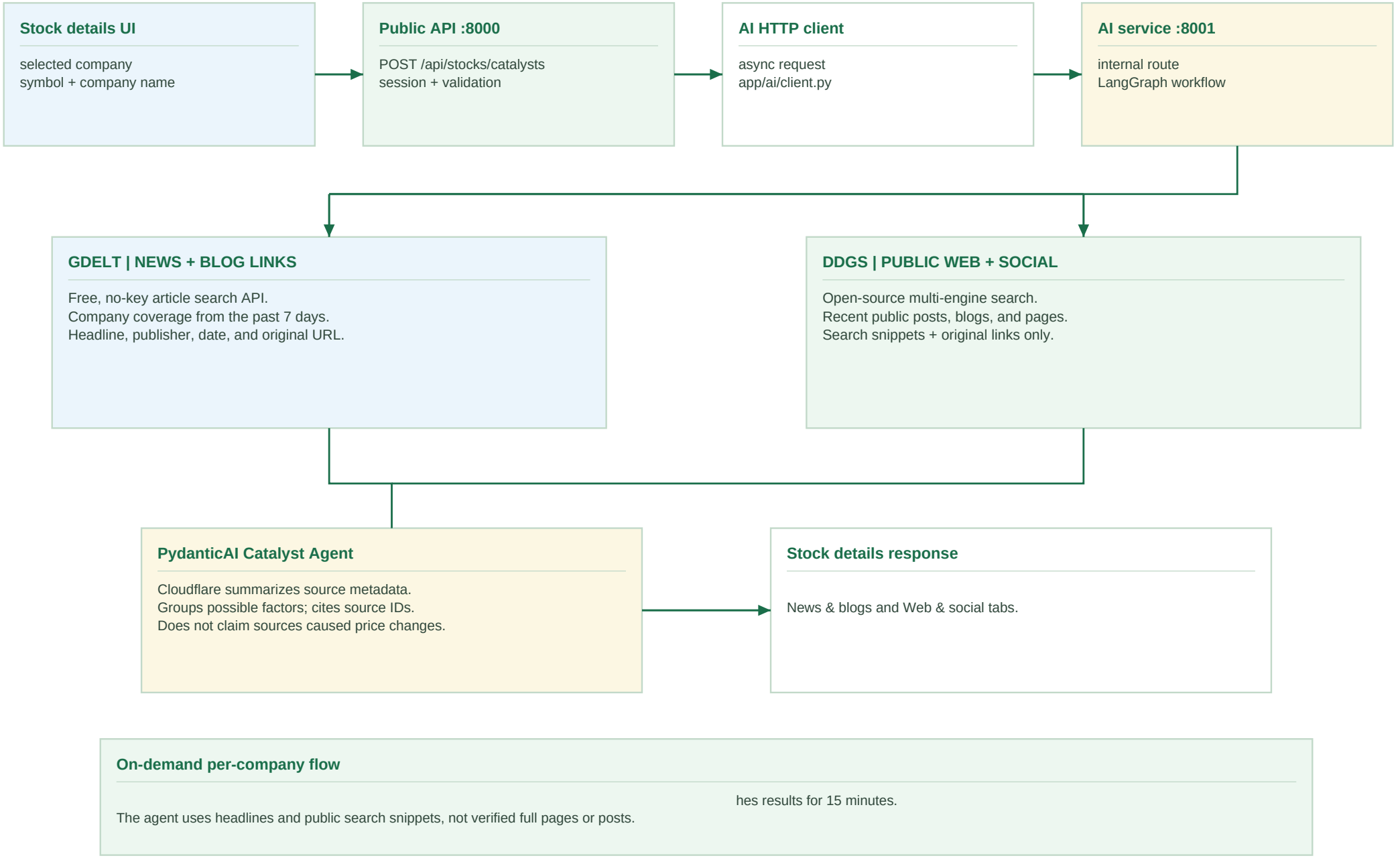
Terminal trace

flow_logger.py assigns a short flow_id.
Boxes mark request, routing, MCP, agent, response.
Use the same ID to follow one request.
Secrets and full prompt data are not logged.

Example analysis trace

REQUEST RECEIVED -> LANGGRAPH ROUTING -> MCP TOOL CALL -> MCP DATA READY
-> STOCK AGENT -> API RESPONSE

Quick debugging path

If no response arrives, check the matching flow_id in the terminal. MCP failures point to the NSE/jugaad-data path; STOCK AGENT failures point to the Cloudflare model or structured output.

LANGGRAPH | MARKET CATALYST AGENT